

Stochastic Robot Behaviors through Hidden Markov Models with Inputs and Outputs

Jesus Savage¹, David A. Rosenblueth^{2*},
Stan Muñoz Gutiérrez³, Marco Negrete¹

¹Bio-Robotics Laboratory, National Autonomous University of Mexico,
Av. Universidad 3000, Coyoacan, Ciudad de Mexico, 04510, Mexico.

²Instituto de Investigaciones en Matematicas Aplicadas y en Sistemas,
National Autonomous University of Mexico, Apdo. 20-126, Ciudad de
Mexico, 01000, Mexico.

³Institute of Software Technology, Graz University of Technology,
Inffeldgasse 16b/II, Graz, 8010, Austria.

*Corresponding author(s). E-mail(s): drosenbl@unam.mx;

Contributing authors: robotssavage@gmail.com;

munozgutierrez@tugraz.at; marco.negrete@ingenieria.unam.edu;

Abstract

Hidden Markov models (HMMs) have found myriad uses. When employed in robotics, however, a conspicuous difference compared with finite-state machine emerges: HMMs lack a direct way of incorporating inputs. Although a sequence of observations can be viewed as input, we argue that it is desirable to extend HMMs with inputs in the vein of finite-state machines. As robot controllers, HMMs exhibit another salient feature: such models primarily focus on capturing uncertainty and stochasticity in the operational environment while maintaining deterministic actions based on the robot’s belief state. We propose extending HMMs in this other direction as well, embracing uncertainty also in the reactive behavior of the robot, resulting in stochastic behavior, thus making the robot act differently under the same conditions. We argue that this produces rich information, often sufficient for effective navigation without explicit maps. In addition, this approach significantly reduces the computational burden on the agent and simultaneously mitigates undesirable behaviors arising from deterministic policies under environmental perception uncertainty. An *extended HMM* (EHMM) can be regarded as a Moore machine with stochastic inputs and outputs. We build EHMM robot behaviors using a genetic algorithm to find the best models and compare this approach with robot behavior implemented with a Markov

decision process (MDP), whose parameters are also found by a genetic algorithm. Our method outperforms MDPs in cases where obstacles are large or the robot is used for exploration.

Keywords: Robot Behaviors, Hidden Markov Models, Markov Decision Processes, Evolutionary Algorithms

1 Introduction

Hidden Markov models (HMMs) have a plethora of applications. When trying to use an HMM to control the navigation of a robot, however, a conspicuous hurdle appears: there is no *direct* way of incorporating the measurements of the sensors. The reason is that HMMs lack *inputs* as they occur, for example, in Mealy or Moore finite-state machines. To be sure, the observations of an HMM can be viewed as inputs of a stochastic process trying to guess the behavior of another, hidden, stochastic process. We argue, however, that it is advantageous to add inputs to the hidden stochastic process of an HMM in the style of Mealy or Moore machines. Another drawback HMMs have as robot navigation controllers is the fact that when faced by the same environment, HMMs exhibit the same behavior. In the context of exploring novel environments, we argue that it is desirable for the robot to exhibit a varying behavior under similar conditions. Our objective is to extend ordinary HMMs in both these directions: we add inputs as well as stochastic outputs, resulting in *extended HMMs*, or EHMMs.

Within the realm of mobile robotics, navigation is a key behavior often implemented using finite-state machines [1, 2]. A finite-state machine can be implemented, for instance, as a Mealy machine [3] (where the output is determined both by the current state and the current input) or as a Moore machine [4] (where the output is determined only by the current state), both of which have inputs.

The complexity of the real world, however, makes probabilistic approaches to robotics [5, 6, 7], such as HMMs, preferable. A drawback of HMMs is that such models only learn the output distribution [8]. All the same, it could be argued that it is possible to view a given observation sequence as input by computing the state sequence that best explains the observation sequence. The nature of such inputs, however, is different from that of Mealy or Moore machines. We modify standard HMMs by incorporating inputs in the spirit of finite-state machines.

Another shortcoming of HMMs when used for controlling robots is the fact that outputs are deterministic. A stochastic behavior, by contrast, would make the robot act differently under the same conditions. We argue that this would produce rich information, often sufficient for effective navigation without explicit maps.

Markov decision processes (MDPs) are also a well-known and widely used technique for robot navigation when there is uncertainty in the execution of movements. If uncertainty is also present in sensor readings so that the current robot state is not fully observable, then Partially Observable MDPs (POMDPs) are a common solution. Hence, we must devote some words to (PO)MDPs. Although these techniques have

shown several advantages, they present drawbacks when compared to our proposal: First, once a policy is trained, robot behavior becomes deterministic. Second, the quality of the derived policy strongly relies on the design of the reward function. Finally, as the number of states grows, calculating the expected reward becomes a task with high computational cost (PSPACE-hard [9] for POMDPs).

This contribution improves previous work [10] by basing EHMMs on Moore machines, as opposed to Mealy machines. The work in [10] uses a matrix C for generating outputs as a function of states and inputs. Such an approach is unsatisfactory, as the genetic algorithm for learning the model requires many iterations to find an optimal C because C is large. This contribution, by comparison, uses two matrices: I , consisting of input probabilities and used for finding the next state, and U , used for generating outputs, which only depends on the current state.

The remaining of the paper is structured as follows. After presenting related work in 2, we review Moore FSMs and HMMs in Section 3. Section 4 describes our extended version of HMMs. We devote Section 5 to the implementation of robot behaviors with EHMMs. Section 6 explains the creation of navigation behaviors using MDPs. The optimization of parameters of (a) probabilistic finite-state machines (PFSMs) based on EHMMs and (b) MDPs for mobile robots behaviors using genetic algorithms (GAs) is in Section 7. We proceed, in Section 8, with the experiments and results. This contribution finishes with a summary of results in Section 9 and concluding remarks in Section 10.

2 Related work

Bengio and Frasconi introduced a model called Input/Output HMM (IOHMM) [8, 11], which is used to learn how to map input sequences to output sequences. This differs from standard HMMs, which focus on learning the distribution of output sequences. These authors studied a discrete-state dynamical system modeled using a state-space representation, where the next state is determined by a function that considers the current state and inputs. The system’s outputs depend on both the current state and the inputs. The system equations describe a generalized Mealy finite-state machine, where both inputs and outputs have continuous values. In their papers, they explore a probabilistic version of these dynamics, using the current inputs and state distribution to predict the state and output distributions for the next time step. The architecture presented in their papers differs from traditional HMMs in two key aspects: computation and learning. In IOHMMs, sequences are processed in a manner similar to that of recurrent networks, allowing an input sequence to be synchronously converted into an output sequence. This computational approach operates in real time, providing output predictions as the input sequence is processed. In IOHMMs, transition probabilities are conditioned on the input, making them time-dependent and leading to an inhomogeneous Markov chain.

The approach of Bengio and Frasconi differs from ours in several ways. First, they estimate the parameters of a stochastic Mealy state machine, whereas we determine those of a stochastic Moore machine. Additionally, to compute state probabilities, they

utilize the entire input sequence, while we rely solely on the present input to determine the next state using a modified version of the Viterbi algorithm, as described in Subsection 4.1. Another key distinction is their use of input-output sequence pairs for training. In contrast, our method, which aims at modeling robotic movement behaviors, involves randomly setting the robot’s initial and final positions within an environment without specifying the intermediate actions required to reach the destination, as explained in Section 8.

Another work related to ours is that by Zhu [12], where HMMs have also been utilized to model dynamic obstacles for mobile robot navigation helping to avoid collisions. Such a work employs an HMM to describe and predict obstacle movements in dynamic environments. Using these predictions, a trajectory-guided path-planning method selects a collision-free path by evaluating a finite set of possible trajectories. Vision processing techniques are applied to determine the motion state of the obstacles. We, by contrast, use modified HMMs to generate navigation behaviors of mobile robots.

Finally, another use of HMMs related to our contribution has been in the bioinformatics field, in which various machine learning techniques have been used to predict protein secondary structures, making this problem one of the most extensively studied challenges in this field. As a result, it serves as a stringent test for the methods proposed by Won et al. [13]. These authors use HMMs to address this problem and determine the model’s parameters using genetic algorithms. Their results outperform the best manually designed HMMs and are competitive with the most advanced predictors developed for this task. Our approach also uses genetic algorithms to find the parameters of our EHMMs.

3 Technical preliminaries

EHMMs can be viewed as an elaboration both of Moore finite-state machines and HMMs. We therefore first cover Moore finite-state machines and then proceed to review HMMs.

3.1 Moore finite-state machines

AFSMs can be built using *deterministic finite-state machines*, such as Moore FSMs. A *Moore FSM*, $\Omega = (S, s_0, \Sigma, \Psi, \gamma, \eta)$, is composed of: a finite set S of states, an initial state $s_0 \in S$, a finite set Σ of input symbols, a finite set Ψ of output symbols, a transition function $\gamma : S \times \Sigma \rightarrow S$, taking a state and an input symbol and returning a state, and an output function $\eta : S \rightarrow \Psi$, mapping each state to an output symbol.

Fig. 1 shows a state diagram of a Moore FSM. This machine has five states, $S = \{s_1, s_2, s_3, s_4, s_d\}$, four input symbols $\Sigma = \{X, \underline{X}, Y, \underline{Y}\}$, and four outputs symbols $\Psi = \{Z_1, Z_2, Z_3, Z_4\}$. For conciseness, the “dead” state s_d and transitions going to s_d are omitted in the figure.

There are several ways to implement a Moore FSM, for example, as an FPGA or with software running on a microprocessor. A drawback, however, is that the stochastic nature of the input signals may produce undesirable behaviors. For example, when the machine in Fig. 1 is in state s_1 , it may be possible that instead of the sensor for

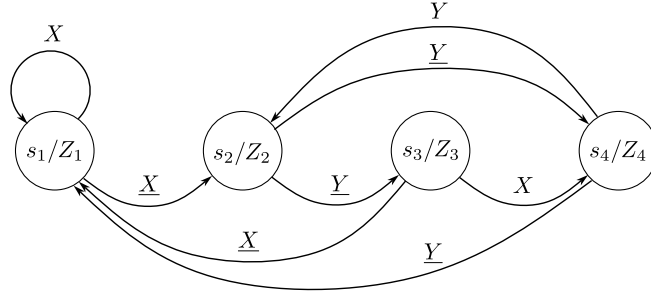


Fig. 1 A state diagram of a Moore FSM.

X getting activated, the sensor for Y is. In this case, the machine will jump to the dead state s_d and will remain in that state until a reset occurs. Moreover, “hazards” may also cause the machine to go to an invalid state. When the machine in Fig. 1 is in state s_1 , for instance, a hazard may make it jump to states s_3 or s_4 .

Yet another disadvantage of a Moore FSM is that under the same values of the inputs Σ , the FSM always behaves the same way, because the machine is deterministic. While this type of behavior is desirable in some applications, in certain robotics scenarios, it may be preferable for the robot to behave differently each time under similar conditions.

All these are reasons for considering a PFSM, able to handle the stochastic nature of input variables and the transition to other states.

3.2 Hidden Markov models

A stochastic algorithm executed by a PFSM depends on random variables [5] and can therefore be modeled using a modified version of HMMs. An HMM is a doubly stochastic process where the set $S = \{s_1, \dots, s_N\}$ of hidden states represents the underlying dynamics of the system and is not directly observable. Instead, there is another set $V = \{v_1, \dots, v_M\}$, of observable symbols. The behavior of an HMM is governed by transition probabilities (between states in S) and emission probabilities (from states in S to symbols in V). The stochastic process exhibits the Markov property, that is, the probability of reaching a particular next state depends only on the current state. The transition probability defines the probability of going from a state to another state or to itself: $a_{ij} = P(S_{t+1} = s_j | S_t = s_i)$, where $S_{t+1} = s_j$ means that at time $t + 1$ the model is in state s_j , and $S_t = s_i$ that at time t the model is in state s_i , that is, a_{ij} is the probability of going from state s_i to state s_j . The probability of emitting symbol v_i being in state s_j is $b_{ji} = P(O_t = v_i | S_t = s_j)$, where $O_t = v_i$ means that the observed symbol at time t is v_i , and $S_t = s_j$ means that the model is in state s_j at time t . The initial state is given by a probability distribution: $\pi_i = P(S_1 = s_i)$, $1 \leq i \leq N$.

More concisely, a *standard HMM* is a tuple $\lambda = (A, B, \pi)$, where A is the state-transition probability distribution matrix (a_{ij}) , $1 \leq i, j \leq N$, B is the observation-symbol probability distribution matrix (b_{ik}) , $1 \leq i \leq N$ and $1 \leq k \leq M$, and $\pi = (\pi_i)$, $1 \leq i \leq N$ is the initial probabilities vector.

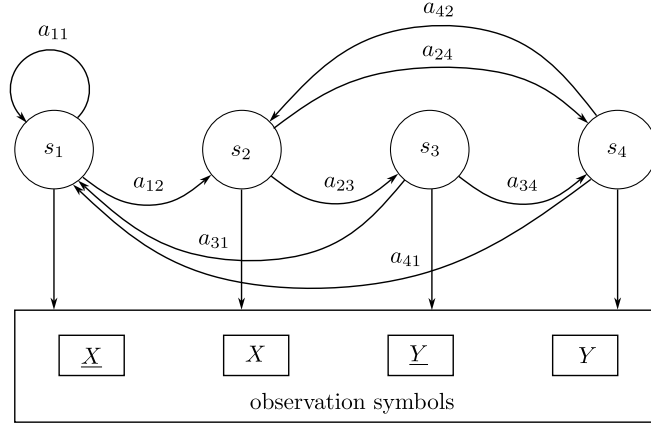


Fig. 2 HMM diagram.

Table 1 Transition probability matrix A

s_i	$a_{i,1}$	$a_{i,2}$	$a_{i,3}$	$a_{i,4}$
s_1	0.49	0.49	0.01	0.01
s_2	0.01	0.01	0.49	0.49
s_3	0.49	0.01	0.01	0.49
s_4	0.49	0.49	0.01	0.01

Fig. 2 shows an HMM whose observation symbols $\{\underline{X}, X, \underline{Y}, Y\}$ are the input symbols of the Moore FSM in Fig. 1. Tables 1 and 2 respectively exhibit the transition probability of matrix A and the observations probability matrix B of this HMM. Hence, the generated observation symbols $O = (O_1, O_2, \dots, O_t)$ can be used as the inputs of a Moore FSM to simulate an environment and therefore test the fitness or performance of such a machine. There is, however, another way to view an HMM: these observation symbols $O = (O_1, O_2, \dots, O_t)$ can be used for another HMM to find the best sequence of hidden states $(S_1, S_2, \dots, S_{t-1})$ using the Viterbi algorithm [14, 15] (summarized in Subsect. 4.1). An HMM can therefore be seen as having two different functionalities: one to generate symbols as outputs and another one to process inputs, observation symbols, and identify them to find the hidden states S . Thus, to implement an FSM using a stochastic state machine it is necessary to modify an HMM to be able to accept inputs and to generate outputs at the same time, as well as to find the next state S_t . The result is an extended version of an HMM, or EHMM.

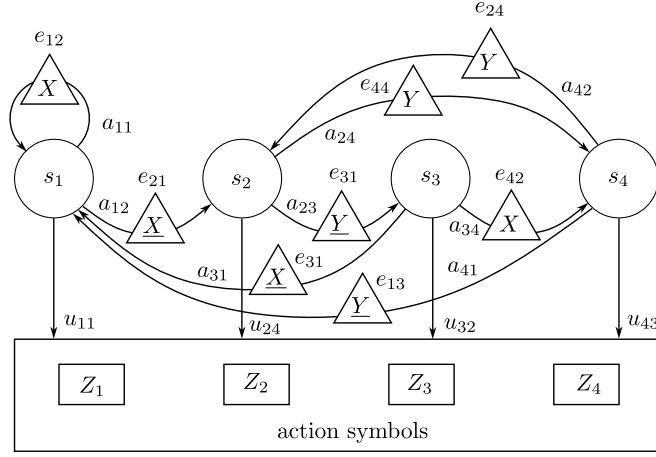
4 Extended hidden Markov models

Our motivation is to use an extended HMM (EHMM) to implement a Moore machine with stochastic inputs, outputs, and transitions from one state to another.

Table 2 Observations probability matrix B

Observation symbols	$v_1 = \underline{X}$	$v_2 = X$	$v_3 = \underline{Y}$	$v_4 = Y$
i	$b_{i,1}$	$b_{i,2}$	$b_{i,3}$	$b_{i,4}$
1	0.45	0.45	0.05	0.05
2	0.05	0.05	0.45	0.45
3	0.45	0.45	0.05	0.05
4	0.05	0.05	0.45	0.45

The state diagram illustrated in Fig. 1 can be transformed into a stochastic state diagram, as depicted in Fig. 3, with a graphical representation akin to that for HMMs. In this graphical representation, we introduce a new symbol, a triangle, representing the stochastic nature of the inputs Σ_t at time t . The stochastic input symbols in this example are $\Sigma = \{\underline{X}, X, \underline{Y}, Y\}$, with the probabilities of the inputs symbols e_{jk} . We can compute the next state using these probabilities and the transition probabilities a_{ij} , as explained in Subsection 4.1.

**Fig. 3** EHMM version for the state diagram shown in 1.

In our proposal, EHMMs, the states are equal to those of an HMM, that are related to the states of the Moore FSM, but taking into consideration the stochastic transition between states with a transition matrix A , similar to the matrix A of an HMM.

Matrix A for the EHMM in Fig. 3 is shown in Table 1.

4.1 Inputs of an EHMM

One way to find the best sequence of states $(S_1, S_2, \dots, S_t)$ that reaches a particular state given a sequence of observations $(O_1, O_2, \dots, O_t)$ with an HMM model λ is by using the Viterbi algorithm [14, 15]. The Viterbi algorithm keeps track of the best

paths, $p_j, 1 \leq j \leq N$, to reach each state given the observations, and uses a variable to keep the probabilities of each path [15]:

$$\delta_t(j) = \max_{1 \leq i \leq N} P(S_1, \dots, S_{t-1} = s_i, O_1, O_2, \dots, O_t | \lambda), 1 \leq j \leq N$$

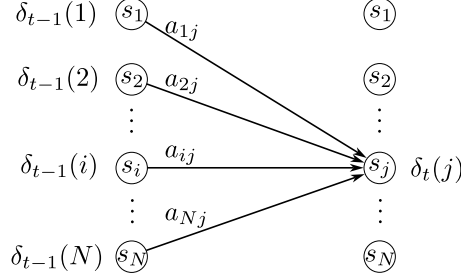


Fig. 4 Best next state s_j given by $\delta_t(j)$.

That is, $\delta_t(j)$ keeps the highest probability of the path to reach state s_j , and this calculation is done for all states $s_i, 1 \leq i \leq N$. Fig. 4 shows each state having a $\delta_{t-1}(j)$ at time $t - 1$ and then at time t , for each state j . The Viterbi algorithm uses the probability of emitting symbol v_i being in state $s_j, b_{ji} = P(O_t = v_i | S_t = s_j) = b_j(O_t)$, at time t to find the value of $\delta_t(j)$:

$$\delta_t(j) = \max_{1 \leq i \leq N} [\delta_{t-1}(i) a_{ij}] b_j(O_t), \quad 1 \leq j \leq N \quad (1)$$

In a deterministic state machine, the next state is obtained by the function $\gamma(S_{t-1}, \Sigma_t)$ that has two arguments: the previous state S_{t-1} and the input Σ_t . In a probabilistic state machine, by contrast, given that the system has been in states $(S_1, S_2, \dots, S_{t-1})$, and with the past input sequence $(\Sigma_1, \Sigma_2, \dots, \Sigma_{t-1})$, where an input $\Sigma_l = \sigma_k$, at time $t = l$, comes from one of the inputs of the set $\Sigma = \{\sigma_1, \sigma_2, \dots, \sigma_K\}$, with a new input Σ_t , the goal is to find the most probable next state s_j at time t . To find the next state from state s_k , we must compute the maximum transition probability of going from any state s_k to s_j .

The main difference of our approach, compared with the Viterbi algorithm, is that we want to decide which is the most probable next state given an input Σ_t and not which is the most probable path p_j :

$$S_t = s_j = \arg \max_{1 \leq i \leq N} [\delta_t(i)]$$

Variable $\delta_t(j)$ must be computed considering the transition probabilities a_{ij} and the input probabilities e_{jk} , where $e_{jk} = P(S_t = s_j | \Sigma_t = \sigma_k) = e_j(\Sigma_t)$ is the probability of observing input symbol $\Sigma_t = \sigma_k$ at time t to reach state S_j .

Table 3 Input Probability Matrix I

Input symbols	$\underline{X}$	X	$\underline{Y}$	Y
State j	$e_{j,1}$	$e_{j,2}$	$e_{j,3}$	$e_{j,4}$
1	0.33	0.33	0.33	0.01
2	0.40	0.10	0.10	0.40
3	0.10	0.10	0.70	0.10
4	0.10	0.40	0.10	0.40

$$\delta_t(j) = \max_{1 \leq i \leq N} P(S_1, \dots, S_{t-1} = s_k, \Sigma_1, \dots, \Sigma_t | \lambda), 1 \leq j \leq N$$

Then, $\delta_t(j)$ can be calculated recursively as follows:

$$\delta_t(j) = \max_{1 \leq i \leq N} [\delta_{t-1}(i) a_{ij}] e_j(\Sigma_t), \quad 1 \leq j \leq N \quad (2)$$

Observe that equation (2) uses the probability $e_j(\Sigma_t)$ instead of $b_j(O_t)$ (as happens in equation (1)), because we are not interested in finding which is the symbol emitted in state s_j . Instead, we wish to find which is the input symbol Σ_t that makes the EHMM reach s_j from any other state s_i . For example, in Fig. 3 there are two ways to reach state s_2 , one is from state s_1 , with the input symbol $\underline{X}$, and another one from state s_4 , with the input symbol Y . Thus, the row of inputs probability matrix I corresponding to state s_2 must reflect the fact that we are expecting to see input variables $\underline{X}$ and Y with high probabilities, as is shown in the second row of Table 3: 0.40 0.10 0.10 0.40.

The inputs probability matrix $I = (e_{jk})$ is used in equation (2), that is, considering the inputs Σ_t used to reach a specific state S_t . The complete input matrix I for the EHMM in Fig. 3 is shown in Table 3.

Matrix I is used to find the next state, given the present state, as expressed in (2). Equation (2) and matrix I are contributions of our research.

4.2 Outputs of an EHMM

We incorporate the outputs of the FSM as action symbols attached to the states of the HMM. The action symbols are $(o_1, o_2, o_3, o_4) = (Z_1, Z_2, Z_3, Z_4)$.

We also incorporate an output matrix U , that contains the probabilities of generating an action $O_t = o_k$ given that the system is in state $S_t = s_j$; hence, $u_{j,k} = p(O_t = o_k | S_t = s_j)$. Thus, matrix U for the EHMM in Fig. 3 is shown in Table 4. For example, the probability to generate output W in state s_1 is $u_{1,1} = 0.85$ in matrix U .

By incorporating the outputs of a deterministic FSM as action symbols in an EHMM, we add a stochastic reactive behavior of the robot, resulting in stochastic behavior, thus making the robot act differently under the same conditions. Matrix U is also a contribution of our research.

Table 4 Action probability matrix U

Output symbols	Z_1	Z_2	Z_3	Z_4
State j	$u_{j,1}$	$u_{j,2}$	$u_{j,3}$	$u_{j,4}$
1	0.85	0.05	0.05	0.05
2	0.05	0.05	0.05	0.85
3	0.05	0.85	0.05	0.05
4	0.05	0.05	0.85	0.05

Also included in the EHMM is the initial state distribution probability:

$$\pi_i = P(S_1 = s_i), \quad 1 \leq i \leq N$$

For the EHMM shown in Fig. 3, the initial probabilities vector is:

$$\pi = (\pi_1, \pi_2, \pi_3, \pi_4) = (0.97, 0.01, 0.01, 0.01)$$

Then, an *Extended Hidden Markov model, or EHMM, is a tuple*

$$\lambda = (A, I, U, \pi)$$

where A , I , and U are matrices, with $A = (a_{ij})$, with N states, for $1 \leq i, j \leq N$; $I = (e_{jk})$, $1 \leq k \leq K$, with K input symbols; $U = (u_{il})$, $1 \leq l \leq L$, with L action symbols; and $\pi = (\pi_i)$.

4.3 Execution of probabilistic algorithm using PFSM

To execute a probabilistic algorithm using a PFSM given its EHMM, $\lambda = (A, I, U, \pi)$, the first state at time $t = 1$, S_1 , is computed with the input observation Σ_1 :

$$\delta_1(i) = \pi_i \cdot e_i(\Sigma_1), \quad 1 \leq i \leq N,$$

$$S_1 = s_j = \arg \max_{1 \leq i \leq N} [\delta_1(i)]$$

Once the first state is computed, the output action can be obtained. Because of the stochastic nature of the probabilistic state machine, the output is not just generated using the action symbol with the highest probability in that state. Instead, the output is obtained by generating a random number and finding the region in which such a number falls. In our approach, the row j of matrix U , corresponding to the current state, is used in the calculation of the probabilities of the output state. With this row, a range of the outputs symbol probabilities is created, as shown in Table 5.

Subsequently, a random number x with uniform density between 0 and 1 is generated. Next, using Table 5 it is checked in which range this number is. If $0 \leq x < r_1$, then the first action output is $AC_1 = U_1$; otherwise, $r_i \leq x < r_{i+1}$, and the first action output is $AC_1 = U_{i+1}$.

Table 5 Symbol output range table.

State i	Range	r
1	$[0, u_{j,1})$	r_1
2	$[u_{j,1}, u_{j,1} + u_{j,2})$	r_2
$\vdots$	$\vdots$	$\vdots$
k	$[\sum_{m=1}^{k-1} u_{j,m}, \sum_{m=1}^k u_{j,m})$	r_i
$\vdots$	$\vdots$	$\vdots$
K	$[\sum_{m=1}^{K-1} u_{j,m}, 1]$	r_K

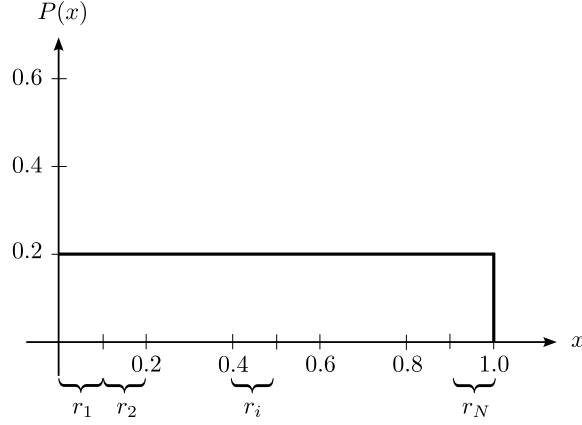


Fig. 5 Probabilistic regions generated with the values of the j row of matrix U of the present state s_j .

After the first state S_1 and the first output AC_1 are calculated, the next state $S_t = s_j$ can be calculated recursively. This procedure continues for $t > 1$ to find the most probable next state:

$$\delta_t(j) = \max_{1 \leq i \leq N} [\delta_{t-1}(i) a_{ij}] e_j(\Sigma_t), \quad 1 \leq j \leq N$$

$$S_t = s_j = \arg \max_{1 \leq i \leq N} [\delta_t(i)]$$

and we generate the output AC_t in the same way as described before.

Optimal parameters of the EHMM, $\lambda = (A, I, U, \pi)$, can be found, for example, by any local-search algorithm, such as a GA. In our research, we optimized the model using a GA, as will be explained in Sect. 7.

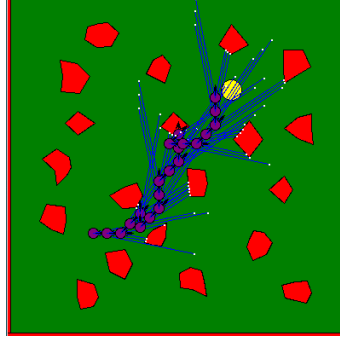


Fig. 6 PFSM tested on a simulated environment.

5 Mobile robot behaviors using PFSMs

Mobile robot behaviors for navigation can be implemented using a PFSM with an EHMM. We tested our approach in a small simulated robot, whose goal is to reach a light source beacon in an environment, while avoiding unknown obstacles in its path, as shown in Fig. 6. In this figure, the free space is drawn in green, the robot is depicted by a purple circle, the obstacles are shown as red polygons, the goal is represented by a yellow circle, and the laser range readings are exhibited as blue lines.

At time t , three sensory values are found: one value is the vector of the range readings, the second value is the position of the light source, and the third value is the intensity of the light source.

The simulated robot had a 240-degree simulated range readings, as does the laser Hokuyo URG_04LXUG01. We selected the readings with a separation of 15 degrees each, making 16 range readings in each sensing at time t , $LR_t = (r_1^t, r_2^t, \dots, r_i^t, \dots, r_{16}^t)$.

Each r_i^t represents the distance from the range sensor to the objects in line of sight at time t , as shown in Fig. 7.

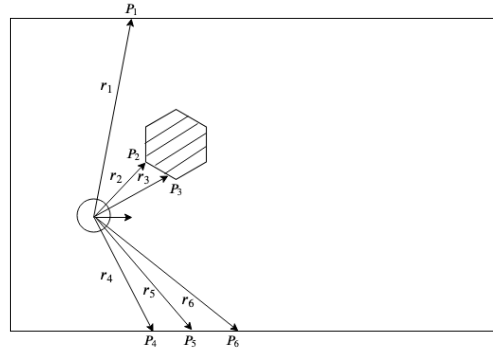


Fig. 7 Mobile robot sensing the environment with 16 range sensors.

The orientation of the light source, ls_t , represented in radians at time t , relative to the robot's center is found. The intensity of light source is li_t .

The data collected at each sensing time t , that is, vector $\underline{LR}_t$, ls_t , and li_t , need to be converted into an input symbol i_t to be used by the EHMM.

5.1 Vector Quantizer

A sensor's quantizer is used to transform sensory data to an index that can be used as the input symbol i_t in an EHMM. A vector quantizer (VQ) is used, which has N centroid vectors that best represent the majority of the sensor readings. The robot navigates in the environment to collect training vectors by taking readings of the orientation, intensity, and range sensors:

$$\begin{aligned}\underline{R}_1 &= (li_1, ls_1, r_1^1, r_2^1, \dots, r_i^1, \dots, r_{16}^1) \\ \underline{R}_2 &= (li_2, ls_2, r_1^2, r_2^2, \dots, r_i^2, \dots, r_{16}^2) \\ &\vdots \\ \underline{R}_n &= (li_n, ls_n, r_1^n, r_2^n, \dots, r_i^n, \dots, r_{16}^n) \\ &\vdots \\ \underline{R}_J &= (li_J, ls_J, r_1^J, r_2^J, \dots, r_i^J, \dots, r_{16}^J)\end{aligned}$$

Given J sensory vectors $\underline{R}_n$, and using the Linde–Buzo–Gray vector quantization algorithm [16], a VQ is found, with a set of centroids $\underline{C} = \{\underline{C}_1, \underline{C}_2, \dots, \underline{C}_N\}$ that best represent training vectors $\underline{R}_n$. This VQ is used to generate an input symbol i_t as follows. The robot gets the sensors' values, obtaining a vector $\underline{R}_t$. To quantize this vector, it is compared with each centroid of the quantizer $\underline{C}$. The centroid vector $\underline{C}_k$ that is the closest vector to $\underline{R}_t$ is chosen. The centroid $\underline{C}_k$ is chosen if:

$$d(\underline{R}_t, \underline{C}_k) < d(\underline{R}_t, \underline{C}_j) \quad \forall j$$

where $d(\underline{R}_t, \underline{C}_j)$ is a function that measures the similitude of the vectors $\underline{R}_t$ and the $\underline{C}_j$ vectors of the quantizer. In this case the similitude function uses the Euclidean distance between vectors. The index k of the centroid vector $\underline{C}_k$ that best represents the sensing vector $\underline{R}_t$ becomes the input symbol $i_t = k$ at time t used by the EHMM.

5.2 Execution of robot behaviors using an EHMM

We now give a summary of the execution of robot behaviors using an EHMM. Note that actions may be different under the same environmental conditions.

Algorithm 1 Robot behavior using an EHMM

Function EHMM Behavior

Input: $\lambda = (A, I, U, \pi)$

Robot's initial pose: $\underline{X}_1 = (x_1, y_1)$

Robot's final pose: $\underline{X}_d = (x_d, y_d)$

Output: Robot's actions: $AC = \{\text{stop, forward, backward, turn left } (45^\circ), \text{ turn right } (-45^\circ), \text{ turn left and go forward, turn right and go forward and turn right twice } (-90^\circ) \text{ and go forward}\}$

$t = 1$

$\underline{R}_1$ = Data collected by the sensors at position $\underline{X}_1$

k = Index of the quantized sensor data $\underline{R}_1$

$\delta_1(i) = \pi_i e_{ik}, \quad 1 < i < N$

$S_1 = s_j = \arg \max_{1 < i < N} [\delta_1(i)]$

$AC_1 = \text{Action}(S_1, U)$

while $\underline{X}_t \neq \underline{X}_d$ **do**

$t = t + 1$

$X_t = \text{Movement}(\underline{X}_{t-1}, AC_{t-1})$

$\underline{R}_t$ = Data collected by the sensors at position $\underline{X}_t$

k = Index of the quantized sensor data $\underline{R}_t$

$\delta_t(j) = \max_{1 < i < N} [\delta_{t-1}(i) a_{ij}] e_{jk}, \quad 1 < j < N$

$S_t = s_j = \arg \max_{1 < i < N} [\delta_t(i)]$

$AC_t = \text{Action}(S_t, U)$

end

6 Mobile-robot behaviors using a Markov decision process

A mobile-robot behavior for navigation using a PFSM with an EHMM can be compared with a behavior implemented using an MDP (e.g., [17]). The MDP finds an optimal solution or policy of movements Π^* , given a set of actions $A(s)$ that the robot can execute; a transition probability matrix $P(s'|s, a)$, where s is the present state, a is an action from $A(s)$ and s' represents the next state; and a reward function $R(s)$.

In this case, the states represent positions of the robot in the environment, because the robot does not have a map of the environment and only sees part of it using its sensors. In this context, our MDP implementation operates episodically, deriving an optimal policy based on its local perception rather than depending on a global map to complete the task. Thus, the states represent only a segment of the complete environment, using a 6×6 squared structure, in which the length of each square is one third of largest measurable distance of the range sensor.

Fig. 8 shows the complete environment where the robot navigates and Fig. 9 shows the observable environment that the robot detects with its sensors.

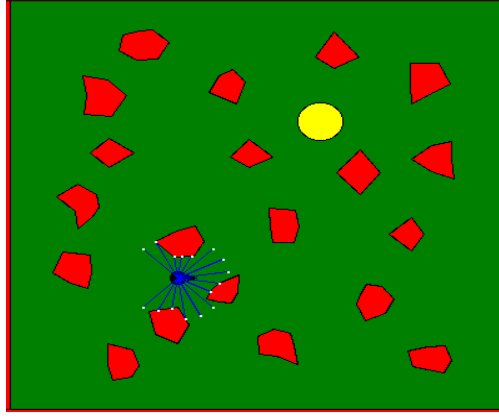


Fig. 8 Complete environment.

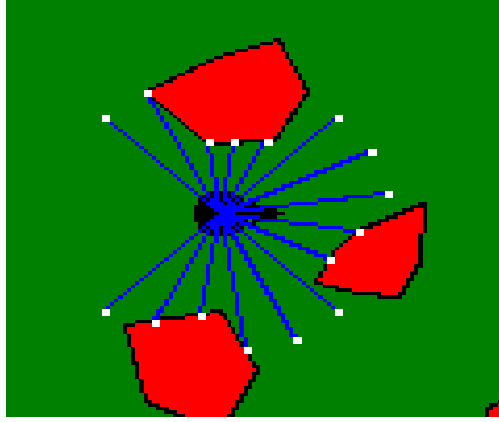


Fig. 9 Partial observable environment.

In each state s the robot receives a reward for its actions. If the robot has not reached the goal, then the reward is -0.04 if there is an obstacle, and -1.0 otherwise. If the robot reaches the state goal, it receives 2.00 .

For the previous observable environment, shown in Fig. 9, the system generates the reward matrix in Table 6, where the robot is positioned in row 3 and column 3. Even though the goal is outside this frame, the goal appears in the upper right part of the matrix.

Each entry may have the values of more than one range sensor.

For each state s , the robot can perform the following actions: $A(s) = \{\text{stop, forward, backward, turn left } (45^\circ), \text{ turn right } (-45^\circ), \text{ turn left and go forward, turn right and go forward and turn right twice } (-90^\circ) \text{ and go forward}\}$.

For each action $A(s)$, there is a transition model probability matrix $P(s'|s, a)$.

Table 6 Reward Matrix

	1	2	3	4	5	6
6	-0.04	⊗	⊗	⊗	-0.04	2.00
5	-0.04	-0.04	⊗	⊗	-0.04	-0.04
4	-0.04	-0.04	-0.04	-0.04	-0.04	-0.04
3	-0.04	-0.04	0.00	-0.04	-0.04	-0.04
2	-0.04	-0.04	-0.04	-0.04	⊗	⊗
1	-0.04	⊗	⊗	⊗	-0.04	⊗

For example, if the robot is in position (2, 2), facing north, and the command is to go forward, Table 7 represents the transition model probability.

Table 7 Transition Model Probability from Going Forward

	1	2	3
3	0.100000	0.700000	0.100000
2	0.050000	0.000000	0.050000
1	0.000000	0.000000	0.000000

There are eight transition model probabilities, corresponding to each action the robot can perform. With these probabilities, the system finds an optimal policy as follows:

$$\begin{aligned}
\Pi_s^* &= \arg \max_{\Pi} U^{\Pi}(s) \\
&= \arg \max_{a \in A(s)} \sum_{s'} P(s'|s, a) U(s')
\end{aligned}$$

where $U(s)$ is the utility function:

$$U(s) = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} P(s'|s, a) U(s')$$

The best $U(s)$ is found solving the Bellman equation via a value-iteration algorithm. Table 8 shows an example of a movement policy Π .

After the robot executes the optimal policy of the first observable environment, it has a maximum of seven steps to reach the local destination. The robot repeats the process of finding the optimal policy given the information provided by the sensors in

Table 8 Movements Policy II

	1	2	3	4	5	6
6	$\searrow$	$\otimes$	$\otimes$	$\otimes$	$\rightarrow$	2.00
5	$\searrow$	$\searrow$	$\otimes$	$\otimes$	$\nearrow$	$\uparrow$
4	$\rightarrow$	$\rightarrow$	$\rightarrow$	$\nearrow$	$\nearrow$	$\uparrow$
3	$\nearrow$	$\nearrow$	$\nearrow$	$\nearrow$	$\nearrow$	$\uparrow$
2	$\nearrow$	$\nearrow$	$\nearrow$	$\nearrow$	$\otimes$	$\otimes$
1	$\nearrow$	$\otimes$	$\otimes$	$\otimes$	$\nwarrow$	$\otimes$

the new observable environment. This process continues until the robot reaches the final goal.

Our proposal is related to, but distinct from, POMDPs, which extend MDPs by incorporating partial observability of states. This feature makes POMDPs well suited for capturing uncertainty in state estimation. While POMDPs are powerful for modeling such uncertainty, they are typically solved to derive a single optimal policy, resulting in identical behavior for identical belief states. This functional approach does not inherently accommodate stochastic actions, limiting built-in exploration capabilities. Solving POMDPs is a PSPACE-complete problem [9], and as such, only approximate solutions are feasible in practice. Although POMDPs are arguably well suited for known environments, the constrained perception capabilities of our agents and the requirement for a low computational footprint make the application of this formalism prohibitive in our domain.

Although reinforcement learning algorithms are frequently preferred for addressing these challenges, our proposed approach diverges by focusing on evolutionary processes. Unlike reinforcement learning, which emphasizes the learning capabilities of individual agents during their operational span, our approach operates at the population scale, evolving stochastic policies across generations rather than within the active timeframe of a single agent.

7 Evolutionary Robotics

Evolutionary robotics (ER) refers to the field of inquiry comprising methods for automatically generating artificial brains and morphologies for autonomous robots that are inspired by Darwinian evolutionary theory [18]. Some applications of ER include [19]:

1. obstacle avoidance [20, 21, 22],
2. wall following [20, 23],
3. object seeking [20],
4. active vision [24], and
5. object detection [25].

Our research is concentrated on how to evolve robot behaviors for object seeking and obstacle avoidance using the concept of probabilistic state machines. To achieve this goal the EHMM robot behavior model, $\lambda = (A, I, U, \pi)$, is evolved, using a GA, to find individuals that are able to look for a light source beacon while avoiding obstacles. A GA performs operations over populations of solutions that resemble sexual reproduction, mutation and artificial selection of apt individuals in a given environment. Often classified within the realm of search and optimization algorithms, it has been argued that GAs are robust techniques that trade off efficacy and efficiency when dealing with different environments by exploiting intrinsic parallelism. The population dynamics allows for the emergent convergence to specific regions of the search space as well as implicit clustering of solutions. GAs have been successfully applied to a variety of engineering and scientific problems, in particular to the evolution of robot controllers and behaviors.

7.1 Evolving Behaviors using EHMMs

For the EHMM, each individual chromosome is composed of: the number of states N_s , the number of input symbols N_l , the number of action symbols N_o , and the number of bits per variable N_b . Then, there are N_s probability variables for vector π , $N_s N_s$ for matrix A , $N_s N_l$ for matrix I and $N_s N_o$ for matrix U . The total number of bits per individual for the EHMM is:

$$N_{EHMM} = N_b(N_s + N_s N_s + N_s N_l + N_s N_o)$$

7.2 Evolving Behaviors using MDPs

For the MDP, each individual's chromosome is composed of: the number of transition probabilities models N_t corresponding to each action that the robot can perform, the number of probabilities for each transition model N_p (e.g., nine according to Table 7), and the number of bits per variable N_b . The total number of bits per individual for the MDP is:

$$N_{MDP} = N_b N_t N_p$$

7.3 ER Methodologies

There are clear methodologies in ER that are mature at present and have diversified over the years [26]. The simplest methodology consists in evolving a set of chromosomes representing the controllers of the robot. Each controller is tested against a simulated environment; selection will grant higher probability of crossover to individuals with higher fitness. Thus, the goal is to use an optimization GA to find the best values of the parameters of a robot behavior to avoid obstacles while the robot attempts to reach a destination.

1. First, a population is randomly generated, with L individuals $B_1, B_2, \dots, B_L$, in which each individual's chromosome is a string of binary numbers representing a behavior $B_i = \{011011 \dots 0101\}$.

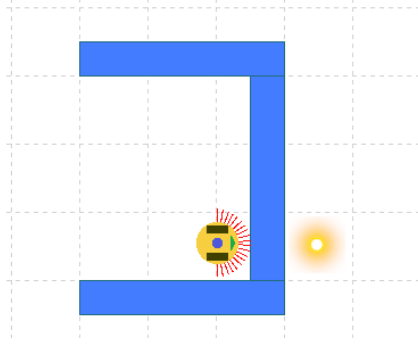


Fig. 10 Even though the robot is close to the destination, the fitness function should consider that the robot must go round the obstacle.

The string is partitioned into small segments that represent the parameters that define the behavior: $B_i = (bh_{i_0} bh_{i_1} \dots bh_{i_j} \dots bh_{i_k})$.

2. Each individual (chromosome) is evaluated giving a fitness value according to the individual's performance. The fitness function evaluates how well the robot's behavior performed its operation, and it takes into consideration several aspects of its performance. Some of them are: the distance between the last position reached and the goal, the number of times the robot hits an obstacle, the number of steps used to reach the goal, and the number of times the robot goes backward. The *Fitness* function used in our experiments is:

$$Fitness = K_1 N D_o + K_2 N / D_d + K_3 S_d \quad (3)$$

The robots have N_k units of time to go from an origin to a destination. N_s is the number of steps that the robot used to reach a destination. If the robot does not reach the destination, then $N_s = N_k$.

In equation (3), $N = (N_k - N_s + 1)$. D_o is the distance between the first position $\underline{X}_o$ and the last position $\underline{X}$ of the robot. That means that a robot that uses few steps to reach the destination and gets faraway from its original position will be well evaluated. D_d is the distance between the last position of the robot and the destination goal, plus the distance to the destination using the shortest path found by the Dijkstra algorithm in a topological map of the environment. This is to make the robot learn that although it may be close to the goal, if there is an obstacle in front of the robot, it needs to go round the obstacle.

$$D_d = \|\underline{X} - \underline{X}_d\| + DistDijkstra(\underline{X}, \underline{X}_d)$$

$DistDijkstra(\underline{X}, \underline{X}_d)$ is a function that measures the distance of the nodes of a topological map that joins the last position of the robot $\underline{X}$ and the destination position $\underline{X}_d$ using the Dijkstra algorithm. The division of N by D_d makes this function reward individuals that get close to the destination. This idea is shown in Figs. 10, 11, and 12.

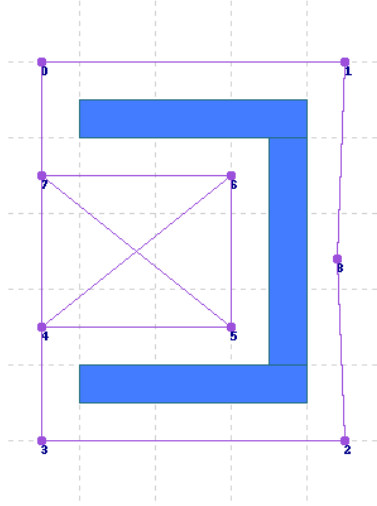


Fig. 11 Environment topological map.

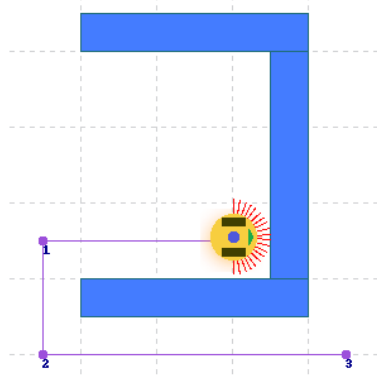


Fig. 12 The best path found by the Dijkstra algorithm is considered so as to find the path that the robot should traverse to reach the destination.

The third term of the *Fitness* function is the standard deviation $\sigma = S_d$, of the robot positions $\underline{X}_i$ between the robot's original position $\underline{X}_o$ and its final position $\underline{X}_d$. This term is used to detect if the robot goes into a local minimum. Constants K_1, K_2 , and K_3 are used to tune the performance of the robot according to its specifications. The values of the constants K_i were found empirically: $K_1 = 10$, $K_2 = 20$, and $K_3 = 11$.

3. The best individuals are selected according to their fitness value. With such selected individuals a new offspring is generated through evolutionary operators (crossover and mutation). The new population consists of the selected parents together with their offspring.

4. Steps 2 and 3 are repeated for either N_g generations or until the difference of the average fitness between two consecutive generations is less than a given value ϵ .

We evolved the individuals using the genetic eclectic algorithm developed by Kuri and Villegas [27]. This algorithm uses deterministic selection and total elitism.

The algorithm presented in Subsect. 5.2 that generated mobile robot behaviors to avoid obstacles was tested in 13 simulated environments, with different sizes, configurations, robot starting positions, and goal destinations. There were two types of environment: one with randomly generated polygons (Fig. 13), and the other one with office-type environments (Fig. 14). Each environment has an area of 4 m^2 , 30 obstacles, and 20 origin/destination points.

For evolving the population, the robot radio was $r = 3 \text{ cm}$, the maximum obstacle radio was $r_o = 16 \text{ cm}$; the radios were fixed during the GA execution. The environments were also surrounded by rectangular walls. We tested the algorithm by simulating the distance of the objects in front of the robot and adding Gaussian noise to such distances. The simulated robot had a 240-degree simulated range readings, similar to the laser Hokuyo URG_04LXUG01, selecting the readings with a separation of 15 degrees each, totaling 16 range readings in each sensing. During the execution of the GA, we added noise both to the movements of the robot and to the sensor. For the linear movement we added a random variable with a uniform density function of $[-0.004, 0.004]$ meters. For rotational movement, we added a uniform density function of $[-0.03927, 0.03927]$ radians. For the range sensor, we added a Gaussian random variable with mean $\mu = 0.004$ and standard deviation $\sigma = 0.0004$.

8 Experiments and Results

8.1 PFSM Tests

Using GAs, the chromosomes representing an EHMM were evolved. The number of individuals per generation was 100, the number of states of the EHMM was $N_s = 16$, the number of input symbols $N_l = 32$, the number of action symbols $N_o = 8$, and the number of bits per variable $N_b = 8$, making a total of:

$$N_{EHMM} = N_b(N_s + N_s N_s + N_s N_l + N_s N_o) = 3,136$$

bits per individual. The mutation factor was equal to 0.1.

Fig. 15 shows how the best individuals evolved during 5,000 generations.

To evaluate the performance of the evolved EHMM, the last optimal generation was tested with two types of environment: office and polygon. We randomly generated 65,000 origin and destination points, and recorded the fitness function of the best individuals. For the office environments, the average performance of the best individuals was 1,345 units, and for the polygon environments the average performance was 3,334 units.

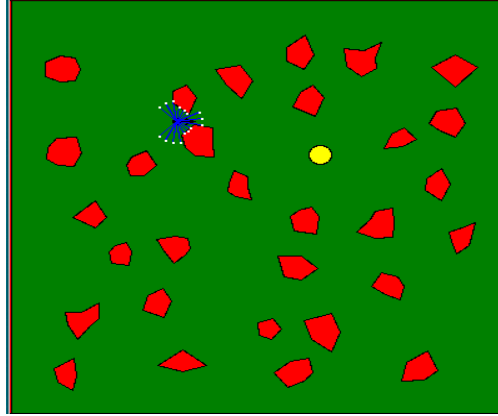


Fig. 13 Environment with polygons generated randomly.

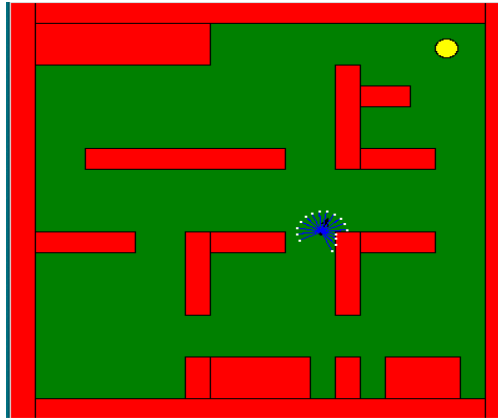


Fig. 14 Office type environment.

8.2 MDP Tests

For the robot behavior using MDPs, the GA found eight transition model probabilities corresponding to each action that the robot can perform. With these models, optimal policies are found. There are $N_t = 8$ transition models, with $N_p = 9$ probability values, and for each probability $N_b = 8$ bits are used, thus each individual was represented by

$$N_{MDP} = N_b N_t N_p = 576$$

bits.

As in the PFSM tests, the population consists of 100 individuals. Fig. 16 shows how the best individuals evolved during 500 generations.

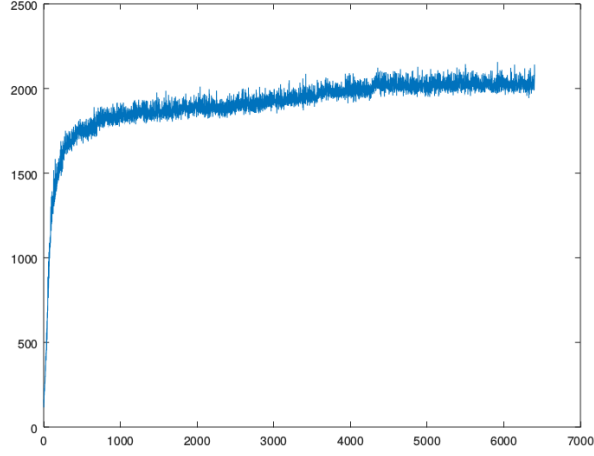


Fig. 15 Best individual EHMM fitness.

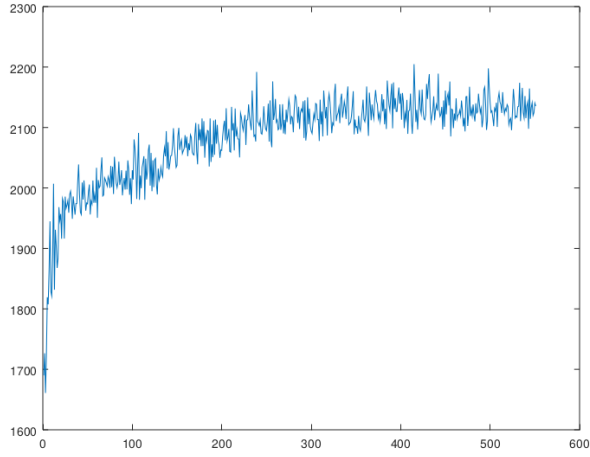


Fig. 16 Best individual MDP fitness.

To evaluate the performance of the evolved MDP, the last optimal generation was tested with two types of environment, with also 65,000 origin and destination points, and the fitness function of the best individuals were recorded. For the office environments, the average performance of the best individuals was 1,272 units, and for the polygon environments the average performance was 3,691 units.

9 Discussion

When the robot is under the same conditions—i.e., the same initial pose x_o, y_o, θ_o and the same destination x_d, y_d , and using a PFSM, it will behave each time in a different way. Figs. 17 and 18 show the execution, under the same conditions, of robot behaviors implemented using EHMM and MDPs. Thus, with the same initial conditions the PFSM finds different paths each time, whereas the MDP behavior always finds the same path. The type of behavior that the EHMM shows can be useful in some robotics applications as in exploration and cleaning areas.

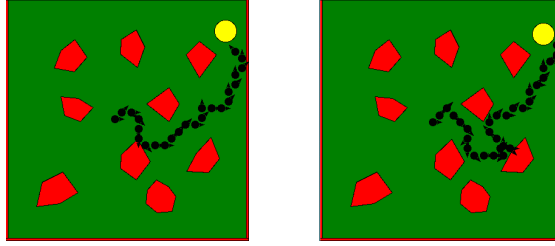


Fig. 17 PFSM executed under the same conditions behaves differently each time.

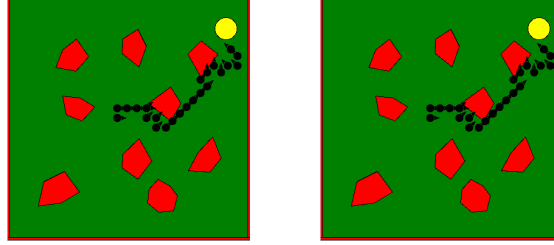


Fig. 18 MDP executed under the same conditions behaves similarly each time.

The main differences between the PFSM and the MDP are the following:

1. The states in the MDP represent the occupancy cell where the robot is in the environment, and in the PFSM the states represent the internal status of the algorithm running in the PFSM.
2. Once the MDP finds the robot movement policy, it can be considered as deterministic. By contrast, the outputs of the PFSM have a stochastic nature, governed by its probabilistic actions.
3. When MDP finds a policy that includes several robot movements and while executing them it does not use its sensors to detect the destination. Thus, if the destination is a light source and if it changes of position, the robot will not notice this, until it finishes the complete execution of the policy. The EHMM senses at every state, and at each state it updates the position of the destination.
4. The EHMM does not need a reward function.

The number of bits for the parameters for the MDP to be optimized by the GA was 576 and for the EHMM 3,136. That means that to have similar performance of the best individuals, the number of generations necessary for the EHMM was approximately 10 times the number of generations than for the MDP. Consequently, the training for the MDP was roughly 10 times faster than for the EHMM. However, during the execution the EHMM was five times faster than the MDP.

10 Conclusions

We proposed a method employing Probabilistic Finite State Machines implemented with Extended Hidden Markov Models to achieve robot behaviors. We compared these robots with robots whose behaviors are implemented instead using Markov Decision Processes. We proposed also how to find the optimal models of this type of EHMM and MDP using genetic algorithms. The main differences between our approach and the classical MDP are that: (a) we do not assume a reward function and (b) the output actions are not deterministic. Our experiments proved that the PFSM outperforms the MDP when they are used for robot navigation in cases when the obstacle is large or the robot is used for exploration.

We envision three avenues for future research. First, just as standard HMMs are equivalent to probabilistic automata with no output probabilities [28, Prop. 8], it would be interesting to find an automaton equivalent to EHMMs. Second, model checking has been developed for HMMs [29, 30]. It would therefore be desirable to develop a temporal logic for EHMMs and a model-checking algorithm thereof. Finally, an algorithm akin to that of Baum–Welsh for learning a model would make EHMMs more relevant.

Declarations

Data availability statement

The code developed in this work is available in a public GitHub repository [here](#).

Funding

This research was supported by the Dirección General de Asuntos del Personal Académico (DGAPA) of the Universidad Nacional Autónoma de México (UNAM), through the Programa de Apoyo a Proyectos de Investigación e Innovación Tecnológica (PAPIIT) grant numbers IT101524, IT102424, and the Programa de Apoyo a Proyectos para Innovar y Mejorar la Educación (PAPIME) PE108624.

Acknowledgments

We would like to thank Fernando Lepe, Carlos Rivera, Andres Buzo, and Federico Kuhlmann, with whom we had fruitful discussions.

Conflicts of interest/Competing interests

There are no conflicts of interest.

Ethics approval

Not applicable.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

Authors' contributions

JS conceived of EHMMs. He also acquired the funds, coordinated the project, did the formal analysis, and wrote the first draft of the manuscript. DAR wrote the subsequent drafts. SMG wrote the MDP software and contributed with general ideas of using MDPs in robot navigation. MN, in turn, contributed with ideas on how to use a genetic algorithm to generate robot behaviors. All authors read and approved the final manuscript.

References

- [1] R. Brooks, A robust layered control system for a mobile robot. *IEEE Journal on Robotics and Automation* **2**(1), 14–23 (1986)
- [2] R.C. Arkin, *Behavior-based robotics* (MIT press, 1998)
- [3] G.H. Mealy, A method for synthesizing sequential circuits. *The Bell System Technical Journal* **34**(5), 1045–1079 (1955)
- [4] E.F. Moore, Gedanken-experiments on sequential machines. *Automata studies* **34**, 129–153 (1956)
- [5] E. Vidal, F. Thollard, C. De La Higuera, F. Casacuberta, R.C. Carrasco, Probabilistic finite-state machines—Part I. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **27**(7), 1013–1025 (2005)
- [6] E. Vidal, F. Thollard, C. De La Higuera, F. Casacuberta, R.C. Carrasco, Probabilistic finite-state machines—Part II. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **27**(7), 1026–1039 (2005)
- [7] S. Thrun, Probabilistic robotics. *Communications of the ACM* **45**(3), 52–57 (2002)
- [8] Y. Bengio, P. Frasconi, An input output HMM architecture. *Advances in neural information processing systems* **7**, 427–434 (1994)
- [9] C.H. Papadimitriou, J.N. Tsitsiklis, The complexity of Markov decision processes. *Mathematics of operations research* **12**(3), 441–450 (1987)
- [10] J. Savage, S. Munoz, L.A. Contreras-Toledo, M. Matamoros, M. Negrete, C. Rivera, G. Steinbabuer, O. Fuentes, H. Okada, *Generating Reactive Robots' Behaviors using Genetic Algorithms.*, in *ICAART (2)* (2021), pp. 698–707
- [11] Y. Bengio, P. Frasconi, Input-output HMM's for sequence processing. *IEEE Transactions on Neural Networks* **7**(5), 1231–1249 (1996)

- [12] Q. Zhu, Hidden Markov model for dynamic obstacle avoidance of mobile robot navigation. *IEEE Transactions on Robotics and Automation* **7**(3), 390–397 (1991)
- [13] K.J. Won, T. Hamelryck, A. Prugel-Bennett, A. Krogh, *Evolving hidden Markov models for protein secondary structure prediction*, in *2005 IEEE Congress on Evolutionary Computation*, vol. 1 (IEEE, 2005), pp. 33–40
- [14] A. Viterbi, Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory* **13**(2), 260–269 (1967)
- [15] L. Rabiner, B. Juang, An introduction to hidden Markov models. *IEEE ASSP magazine* **3**(1), 4–16 (1986)
- [16] Y. Linde, A. Buzo, R. Gray, An algorithm for vector quantizer design. *IEEE Transactions on Communications* **28**(1), 84–95 (1980)
- [17] O. Ibe, *Markov Processes for Stochastic Modeling* (Newnes, 2013)
- [18] S. Nolfi, J. Bongard, P. Husbands, D. Floreano, in *Handbook of Robotics*, ed. by B. Siciliano, O. Khatib, 2nd edn. (Springer, 2016), chap. 76, pp. 2035–2067
- [19] L. König, *Complex Behavior in Evolutionary Robotics* (Walter de Gruyter GmbH & Co KG, 2015)
- [20] W. Banzhaf, P. Nordin, M. Olmer, *Generating adaptive behavior using function regression within genetic programming and a real robot*, in *2nd International Conference on Genetic Programming* (Citeseer, 1997), pp. 35–43
- [21] H.S. Seok, K.J. Lee, B.T. Zhang, *An on-line learning method for object-locating robots using genetic programming on evolvable hardware*, in *5th International Symposium on Artificial Life and Robotics*, vol. 1 (Citeseer, 2000), pp. 321–324
- [22] A.L. Nelson, G.J. Barlow, L. Doitsidis, Fitness functions in evolutionary robotics: A survey and analysis. *Robotics and Autonomous Systems* **57**(4), 345–370 (2009)
- [23] M. Szymanski, L. Winkler, D. Laneri, F. Schlachter, A.C. van Rossum, T. Schmickl, R. Thenius, *Symbricatorrtos: a flexible and dynamic framework for bio-inspired robot control systems and evolution*, in *2009 IEEE Congress on Evolutionary Computation* (IEEE, 2009), pp. 3314–3321
- [24] D. Marocco, D. Floreano, *Active vision and feature selection in evolutionary behavioral systems*, in *7th International Conference on Simulation of Adaptive Behavior* (EPFL, 2002), pp. 247–255
- [25] J.C. Zagal, J. Ruiz-del-Solar, P. Guerrero, R. Palma, *Evolving visual object recognition for legged robots*, in *RoboCup 2003: Robot Soccer World Cup VII* 7 (Springer, 2004), pp. 181–191
- [26] S. Nolfi, D. Floreano, *Evolutionary Robotics: The Biology, Intelligence, and Technology of Self-Organizing Machines* (MIT press, 2000)
- [27] A. Kuri, C. Villegas, *A universal eclectic genetic algorithm for constrained optimization*, in *6th European Congress on Intelligent Techniques and Soft Computing*, vol. 1 (1998), pp. 518–522
- [28] P. Dupont, F. Denis, Y. Esposito, Links between probabilistic automata and hidden Markov models: probability distributions, learning models and induction algorithms. *Pattern recognition* **38**(9), 1349–1371 (2005)
- [29] L. Zhang, H. Hermanns, D.N. Jansen, *Logic and model checking for hidden*

- Markov models*, in *Formal Techniques for Networked and Distributed Systems-FORTE 2005: 25th IFIP WG 6.1 International Conference, Taipei, Taiwan, October 2-5, 2005. Proceedings 25* (Springer, 2005), pp. 98–112
- [30] J. Buckby, T. Wang, J. Zhuang, K. Obara, Model checking for hidden Markov models. *Journal of Computational and Graphical Statistics* **29**(4), 859–874 (2020)